

Prototype Skill — AI Council Pre-Read

Audience: AI Council (security, data, technical governance)

Date: 2026-07-21

Prepared by: Beth Connor, UX Manager, N-able · M.Des + MBA, IIT Institute of Design

Reading time: ~8 min

Ask: Review for security, data, and governance fit.

Overview

What it is: A structured workflow that uses Claude (Anthropic's AI) to help produce UX prototypes — working HTML mockups used to evaluate product concepts before build decisions are made. It runs locally. It produces files, not running systems.

How it works: A 7-step pipeline — pull source material from Confluence and Jira, structure it into an evidence-tagged brief, run analytical review, build a static HTML prototype, review again, run a multi-framework council analysis, and produce a handoff packet. Every step is defined. Every strategic call is left blank for the human.

What it is not: Not code generation. Not vibe coding. It is a specification system written in natural language — closer to a detailed SOP than to software. I defined the pipeline stages, the evidence standards, the quality gates, and the governance rules. The AI executes the steps.

What this tool is — and isn't

What it is: A 7-step pipeline that pulls source material (Confluence, Jira), structures it into an evidence-tagged brief, runs analytical review, and produces a static HTML mockup. It runs on a local machine. It produces files, not running systems.

What it is not:

- Not production code generation
 - Not a tool with elevated access — it reads only what the operator can already read
 - Not autonomous — every strategic call is left explicitly blank for the human
 - Not "vibe coded" — the pipeline has defined stages, hard stops, verification gates, and an audit trail built into every output
-

Security

Bottom line: The risk surface is the same as any N-able employee using Claude Code with their own credentials. This skill introduces no novel attack vectors.

Longer answer:

- The skill runs locally on the operator's machine. No separate server, no custom API endpoint, no data pipeline.

- It calls the Anthropic API (Claude) and, via MCP connectors, Atlassian APIs (Confluence, Jira). Both are already in N-able's approved tool stack.
- The Atlassian MCP connector authenticates as the operator — it uses the same OAuth credentials the person already has. The tool cannot access anything the operator cannot access manually.
- Prototypes are HTML files. They contain no live API calls, no authentication tokens, no connection strings. They are static documents with fictional seed data.
- The skill explicitly prohibits real customer data, real tenant names, and real pricing in prototype seed data. This is enforced by a documented rule ("Pete Hegseth test": if you wouldn't want it public, it does not go in a prototype).

What the skill does NOT do:

- Write to any production system
 - Exfiltrate data to any third party beyond the Anthropic API (standard LLM call)
 - Store session data beyond what Anthropic's standard data retention covers
 - Execute code on remote systems
-

Data

Bottom line: The operator's access scope is the tool's access scope. Nothing more.

When the skill pulls from Confluence or Jira:

- It authenticates as the person running it, using their existing MCP OAuth token
- It can only read pages and tickets that person can already read
- It cannot escalate permissions or read across accounts

What gets sent to the Anthropic API:

- Text content from Confluence pages / Jira tickets the operator retrieves
- The operator's prompts and the skill's pipeline instructions
- Standard LLM inference call — same data handling as any Claude API usage

What stays local:

- The HTML prototype files
- The brief files (`.brief.md`)
- The handoff packet folder structure

Seed data rule: All names, companies, and customer-identifiable data in prototypes must be fictional. The skill's instructions explicitly require this. A reviewer checking any prototype output should see invented names, not real tenant or customer data.

How it works

The pipeline (7 steps)

0. Intake → operator answers 4 questions (lo-fi vs. hi-fi, audience, source docs, council depth)
1. Scout → pulls source material from Confluence, Jira, or local files — raw, no synthesis
2. Spec → structures the material into a three-gate brief with evidence tagging
3. Critic → pre-build scorecard across 6 dimensions; produces a punch list
4. Build → generates working HTML prototype
5. Critic → post-build re-score; delta table vs. pre-build
6. Council → 44-framework analytical review; contradictions preserved, not resolved
7. Handoff → structured packet with verified math, named tensions, decision calls

All 7 steps run. No steps drop even if the operator says "just build it."

On the "vibe coding" question: This is a specification system, not improvised prompting. I defined the pipeline stages, data schemas, quality gates, hard stops, and governance rules in structured natural language — the same intellectual work as writing a technical spec or SOP, in a different medium. The AI executes defined steps. It does not invent the process or make judgment calls. The audit trail is built into every output.

The frameworks underlying this system — structured planning, evidence tagging, attribution methodology — come from graduate-level design research (M.Des + MBA, IIT Institute of Design). I am a subject matter expert applying trained methodology to a new medium, not a non-technical user improvising with AI.

Evidence tagging — how claims are tracked

Every factual claim in the brief and output is tagged:

Tag	Meaning
[R]	Research — from a primary source (interview, study, document)
[D]	Data — quantitative, from a named dataset
[A]	Assumed — not verified, stated as working assumption
[?]	Unknown — gap explicitly named

If [A] + [?] claims exceed 30% of world-claims in a brief, the brief is flagged as not ready. The readiness score in the prototype reflects this honestly — the tool does not paper over thin evidence with a confident score.

Attribution — who made what

Every component of the output is tagged with what percentage came from the human operator vs. the AI tool. The attribution system and the council both use a practice I call **owensing**, derived from Charles Owen's Structured Planning methodology (IIT Institute of Design). Owen's RELATN process maps every solution element against every function — not to find similarities, but to find reinforcement and conflict. The conflict regions are the signal. Functions that share conflicting solutions are the ones that need to be considered together.

Applied here: owensing is the cross-check. In the **attribution system**, it means tracing which judgment came from whom — not as a flat percentage, but as a provenance map. In the **council**, it means mapping which analytical frameworks share recommendations and which conflict on the same point. The council without this cross-check is a list. With it, it's a structured tension map — you can see which

conflicts are structural (multiple frameworks hitting the same fault line) vs. isolated (one framework's edge case). The synthesis slot stays blank because that call is the human's, not the tool's.

This is not an ad hoc invention. It is applied graduate-level design research methodology in a new medium.

Why it exists: The tool produces content that looks authoritative. Owensing makes the human's actual contribution visible so reviewers — and the operator — can see what is "their work" vs. what is AI-generated output they're presenting.

How it's calculated: Three methods (operator picks what fits):

1. **Edit distance** — how much did the human change the tool's output? (100% = wrote from blank page; 0% = accepted verbatim)
2. **Origination** — who brought the idea first? Human seed + tool expanded + human edited $\approx$ 50%
3. **Self-report** — operator states their honest percentage

Where it appears: Every routing card, strategy recommendation, and analysis block in the prototype carries an attribution badge (e.g., "65% Beth"). The synthesis slot in the council summary is always left blank with `[0% Claude - fill this in]`, because the tool does not make strategic decisions.

What it prevents: A PM presenting AI-generated strategy analysis as "my analysis" without disclosure. The attribution is structural — it's built into the output format, not optional.

The council — what "44 voices" means

Step 6 runs the concept through 44 analytical frameworks drawn from design, strategy, behavioral economics, and organizational theory (e.g., Jobs-to-Be-Done, Self-Determination Theory, Value Chain Analysis, Disruptive Innovation).

Each framework ("voice") returns:

- What it sees the concept doing well
- What it predicts will fail
- What it cannot see from its own vantage point

Voices are encouraged to contradict each other. The contradictions are the output — they surface the tensions a single-framework analysis would miss. No voice "wins." The operator reads the tensions and decides what they mean. The synthesis sentence is always blank.

Fast Mode (default): 7 key tensions + executive summary — ~2 min read.

Hard Mode (opt-in): All 44 voices with full cross-talk — ~15–20 min read.

The 47% rule

Any ratio, count, or statistic that appears in a handoff document must be grep-verified against source material before the handoff ships. The rule exists because the tool once self-reported a 47% figure in a brief; a grep of the actual source showed 80%. The tool's fluency does not track its own math.

Verification is enforced as a named pipeline step (Bradley verification gate) before handoff.